

Computing the partial conjugate of convex piecewise linear-quadratic bivariate functions

Bryan Gardiner · Khan Jakee · Yves Lucet

Received: 22 June 2011 / Published online: 3 December 2013
© Springer Science+Business Media New York 2013

Abstract Piecewise linear-quadratic (PLQ) functions are an important class of functions in convex analysis since the result of most convex operators applied to a PLQ function is a PLQ function. We modify a recent algorithm for computing the convex (Legendre-Fenchel) conjugate of convex PLQ functions of two variables, to compute its partial conjugate i.e. the conjugate with respect to one of the variables. The structure of the original algorithm is preserved including its time complexity (linear time with some approximation and log-linear time without approximation). Applying twice the partial conjugate (and a variable switching operator) recovers the full conjugate. We present our partial conjugate algorithm, which is more flexible and simpler than the original full conjugate algorithm. We emphasize the difference with the full conjugate algorithm and illustrate results by computing partial conjugates, partial Moreau envelopes, and partial proximal averages.

Keywords Legendre-Fenchel transform · Convex conjugate · Piecewise linear-quadratic functions · Computational convex analysis · Partial conjugate

1 Introduction

Convex analysis is a broad field with applications in many areas in engineering and the sciences. It provides a wealth of operators for working with convex functions. Computational convex analysis (CCA) focuses on designing efficient algorithms to compute such operators. CCA began with the development of the fast algorithms for computing the Legendre-Fenchel transform [7, 12, 26, 34, 44], and has since encompassed many other operators [29]. We refer to [30] for a survey of CCA and a

B. Gardiner · K. Jakee · Y. Lucet (✉)
Computer Science, I. K. Barber School, University of British Columbia Okanagan, Kelowna,
British Columbia V1V 1V7, Canada
e-mail: yves.lucet@ubc.ca

list of several applications of these algorithms in a wide variety of fields (computer-aided convex analysis, network flow, phase transition in thermodynamics, electrical networks, robot navigation, image processing, computer vision, partial differential equations, multifractal analysis, network communications, morphology neural networks, etc.).

Piecewise linear-quadratic (PLQ) functions possess very interesting properties from both the theoretical [41, Chap. 10.E and 11.D] and numerical point of view [31]. They provide insight into the behavior of operators in convex analysis. The set of piecewise linear functions (resp. piecewise linear-quadratic functions) is closed under the conjugacy operation [41, Theorem 11.14]. In addition, the class of piecewise linear-quadratic functions is closed under the Moreau envelope operator [41, Example 11.28]. In applied optimization, piecewise linear approximation is a convenient way to work with a function for which a closed form is complex, or does not exist.

Given a bivariate function as a black box, one can build a PLQ function using a variety of strategies e.g. interpolation with quadratic box splines [13], piecewise Bernstein-Bézier representation [45], Powell-Sabin splines [1, 15, 39, 42, 46, 48], simplex B-splines [8, 43], or approximation [32, 47] (see also [25]). Hence, we assume the approximation or interpolation is already computed and focus on the computation of the conjugate.

Recent results in parametric optimization use PLQ functions and allow the computation of the Moreau envelope of convex PLQ functions, not necessarily bivariate, by solving a parametric quadratic programming problem [35]. The motivation stems from the fact that some optimal control problems can be formulated as parametric optimization problems and solved explicitly offline [36]. The convex parametric piecewise quadratic optimization problem studied in [35–37] allows to compute the Moreau envelope of a convex PLQ function but does not allow one to compute the partial conjugate since the function to maximize is no longer convex due to the introduction of the nonconvex term $(x, y) \mapsto xy$.

A wide range of algorithms for PLQ functions of one variable have been studied [16–18, 31], and are implemented in the Computational Convex Analysis software package [27]. More recently, work has focused on extending these algorithms to PLQ functions of two variables (henceforth referred to as *PLQ2d functions*) [19].

PLQ functions have a simple structure that is easy to represent and manipulate. Unfortunately, many of the techniques used to achieve efficient algorithms in one dimension, e.g. an ordering of the domain [28] or representing the graph of the subdifferential [18], do not work for a function with a two-dimensional domain. However, the domain of a PLQ2d function is a *planar subdivision*, a well-known structure in computational geometry. We can thus draw from that field for useful tools and asymptotic complexity results.

The recent conjugate algorithm for convex bivariate PLQ functions [19] showed that the conjugate can be computed in log-linear time (linear time with some approximation). We focus on computing the *partial conjugate* i.e. the conjugate with respect to only one variable instead of all of them. The partial conjugate is more flexible since the full conjugate can be computed by two partial conjugate computations. Similarities between the full conjugate and the partial conjugate allow a partial conjugate algorithm to be designed following the basic outline of the existing, full conjugate algorithm in [19].

In this paper, we propose a new algorithm to compute partial conjugates. Section 2 introduces notations for this paper. Theoretical results are proved in Sect. 3. We then present several numerical examples in Sect. 4 followed by the description of the algorithm. Finally, we conclude our work in Sect. 5, and discuss future directions.

2 Preliminaries

2.1 Notations

We begin with notations that we will use throughout this paper. We employ the standard extended arithmetic used in Convex Analysis, where $\mathbb{R} \cup \{+\infty\}$ is the set of *extended real numbers*. For vectors $x, y \in \mathbb{R}^2$ we write the inner product either as $\langle x, y \rangle$, or in vector notation as $x^T y$. (All vectors are column vectors, and the notation x^T denotes the transpose of x i.e. the row vector $[x_1, x_2]$.) We denote the quadratic kernel $q(x) = \|x\|^2/2$ where $\|\cdot\|$ is the Euclidean norm, and the indicator function of a set C with

$$\iota_C : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\} : x \mapsto \begin{cases} 0, & \text{if } x \in C; \\ +\infty, & \text{if } x \notin C. \end{cases}$$

The convex hull of a set C (the intersection of all convex sets containing C) will be denoted by $\text{co } C$. The interior (resp. relative interior) of the set C will be noted $\text{int } C$ (resp. $\text{ri } C$). The effective domain of a function f , i.e. the set of all points where f takes a finite value, is noted $\text{dom } f$. A function $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{-\infty, +\infty\}$ is *proper* if it has nonempty domain and is greater than $-\infty$ everywhere.

The (convex) subdifferential of a function $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ is defined as

$$\partial f(x) = \{s : f(y) \geq f(x) + \langle s, y - x \rangle, \forall y \in \mathbb{R}^n\},$$

while the (convex) conjugate of f is the convex function

$$f^* : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\} : s \mapsto \sup_{x \in \mathbb{R}^2} [\langle s, x \rangle - f(x)].$$

When f is convex, f^* is lower semi-continuous (lsc) convex, and proper if, and only if, f is proper [40, Theorem 12.2]. (See [41, Definition 1.5] for the definition of lower semi-continuity.) The *Moreau envelope* (or Moreau-Yosida approximate) defined by

$$e_\lambda f(y) = \inf_{x \in \mathbb{R}^2} \left[f(x) + \frac{1}{2\lambda} \|x - y\|^2 \right] \quad (\lambda > 0)$$

is a smoothing operator with applications in proximal point and interior point methods. Computation of the Moreau envelope and the conjugate are equivalent by the following formulas,

$$\begin{aligned} e_\lambda f(y) &= \frac{\|y\|^2}{2\lambda} - \frac{1}{\lambda} \left(\lambda f + \frac{1}{2} \|\cdot\|^2 \right)^* (y), \\ f^*(s) &= \frac{\|s\|^2}{2} - \lambda e_\lambda \left(\frac{1}{\lambda} f - \frac{\|\cdot\|^2}{2\lambda} \right) (s). \end{aligned} \tag{1}$$

(See [41, Example 11.26(c)] or just expand the norm square for a proof.)

The proximal average is an operator that allows continuous transformation from one function to another [3]. It is defined by

$$\mathcal{P}_\mu(f_0, f_1; \lambda_0, \lambda_1) = \left(\lambda_0 \left(f_0 + \frac{q}{\mu} \right)^* + \lambda_1 \left(f_1 + \frac{q}{\mu} \right)^* \right)^* - \frac{q}{\mu},$$

for proper lsc convex functions $f_0, f_1 : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ where $\mu > 0$ is a smoothing parameter, and $\lambda_0, \lambda_1 > 0$ with $\lambda_0 + \lambda_1 = 1$. It can also be computed as [22, Formula (4)]

$$\mathcal{P}_1(f_0, f_1; \lambda_0, \lambda_1) = -e_1[-\lambda_0 e_1(f_0) - \lambda_1 e_1(f_1)].$$

The proximal average has many useful properties. It is self-dual: the conjugate of the proximal average is the proximal average of the conjugates [2, Theorem 5.1], i.e. $[\mathcal{P}_\mu(f_0, f_1; \lambda_0, \lambda_1)]^* = \mathcal{P}_\mu(f_0^*, f_1^*; \lambda_0, \lambda_1)$. The proximal average also averages the domains of the functions [2, Theorem 4.6]: $\text{dom } \mathcal{P}_\mu(f_0, f_1; \lambda_0, \lambda_1) = \lambda_0 \text{dom } f_0 + \lambda_1 \text{dom } f_1$. We refer to [2–5, 24] for further properties. While the proximal average has been extended to the nonconvex setting in [22], we restrict our present work to the convex case.

Other smoothing operators besides the Moreau envelope include

$$s_\lambda f = (1 - \lambda^2)e_\lambda f + \lambda \frac{\|\cdot\|^2}{2}$$

which is introduced in [21], and

$$T_\lambda f = \mathcal{P}_1(f, q; 1 - \lambda, \lambda)$$

which is studied in [5, 33]. (Both of these operators are self-dual, and require $\lambda \in (0, 1)$.) Our algorithm can be directly applied to compute such operators.

2.2 Piecewise Linear-Quadratic (PLQ) functions

To define *piecewise linear-quadratic (PLQ) functions* $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$, as studied in [41] and implemented in [31] for univariate functions and in [19] for bivariate functions, we first introduce polyhedral sets, faces, edges, and vertices.

Definition 2.1 ([41, Example 2.10]; Polyhedral sets and affine sets) A set $C \subset \mathbb{R}^n$ is said to be a polyhedral set if it can be expressed as the intersection of a finite family of closed half-spaces or hyperplanes, or equivalently, can be specified by finitely many linear constraints, i.e., constraints $f_i(x) \leq 0$ or $f_i(x) = 0$ where f_i is affine. It is called an affine set if it can be expressed as the intersection of hyperplanes alone, i.e., in terms only of constraints $f_i(x) = 0$ with f_i affine.

Affine sets are in particular polyhedral, while polyhedral sets are in particular closed, convex sets. The empty set and the whole space are affine.

Polyhedral sets with dimension 2 (resp. 1, 0) in the plane will be called *faces* (resp. *edges*, *vertices*). In other words, faces are polyhedral sets with nonempty interiors. Furthermore, edges defined as $E = \{x \in \mathbb{R}^2 : x = \lambda_1 x^1 + \lambda_2 x^2\}$ with $x^1 \neq x^2$ and $\lambda_1 + \lambda_2 = 1$ are called either a (bounded) *segment* when $\lambda_1 \geq 0$, $\lambda_2 \geq 0$, a *ray* or half-line when $\lambda_1 \geq 0$ but $\lambda_2 \in \mathbb{R}$, or a *line* in the plane when $\lambda_1, \lambda_2 \in \mathbb{R}$. We refer to vertices, edges, and faces as *entities*.

We can now define PLQ bivariate functions.

Definition 2.2 A PLQ bivariate function is a function $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ whose domain can be represented as the union of finitely many polyhedral sets P_i relative to which the function is quadratic i.e. for $x \in P_i$, $f(x) = 1/2x^T A_i x + b_i^T x + c_i$ with A_i a 2×2 symmetric matrix, $b_i \in \mathbb{R}^2$, and $c_i \in \mathbb{R} \cup \{+\infty\}$.

We will also use the notation $f_i : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ defined by $f_i(x) = x^T A_i x + b_i^T x + c_i$. When $x \in P_i$ we have $f(x) = f_i(x)$. This definition of PLQ functions follows [41, Definition 10.20, p. 440]. Note that at the intersection $P_i \cap P_j$ of two polyhedral sets, the formula for f_i and f_j must agree i.e. for all $x \in P_i \cap P_j$ $f(x) = f_i(x) = f_j(x)$.

For example, the function $f(x_1, x_2) = |x_1| + |x_2|$ on $\mathbb{R}^2$ is a convex bivariate PLQ function. However, the function $f(x_1, x_2) = |x_1^2 + x_2^2 - 1|$ on $\mathbb{R}^2$ is not since its domain cannot be partitioned as a finite number of polyhedral sets on which f is quadratic. The function $f(x_1, x_2) = 0$ if $x_1 \leq 0$ and $f(x_1, x_2) = 1$ otherwise is also not a PLQ function since it is not continuous on its domain.

PLQ functions enjoy several properties summarized in [41, Proposition 10.21] e.g., the domain $\text{dom } f$ of a bivariate PLQ function $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ is closed, f is continuous on $\text{ri dom } f$, and lsc on $\mathbb{R}^2$; moreover when f is convex, $\text{dom } f$ is polyhedral; the subgradient sets $\partial f(\bar{x})$ at any $\bar{x} \in \text{dom } f$ are then polyhedral as well, and $\partial f(\bar{x}) \neq \emptyset$.

Since the class of PLQ functions is closed under most convex operators [41, Exercise 10.22, Example 11.28, Proposition 11.32], a toolbox was built to manipulate convex bivariate functions [19] as summarized in the next section.

2.3 Data structure

Recall that an entity is a set in $\mathbb{R}^2$ which is either a vertex, an edge, or a face. We store a bivariate PLQ function f as a union of polyhedral sets representing $\text{dom } f$ and a list of quadratic functions representing the function on each entity. Each entity is assigned an index i in a finite index set I . For each value of i we store a function

$$f_i(x) = x^T A_i x + b_i^T x + c_i,$$

where $A_i \in \mathbb{R}^{2 \times 2}$ is symmetric, $b_i \in \mathbb{R}^2$, and $c_i \in \mathbb{R} \cup \{+\infty\}$. If $c_i = +\infty$, then the function f is unbounded in that region. (That data structure is more general since it can store bivariate functions that are not PLQ like the function $f(x_1, x_2) = x_1$ when $x_1 < 0$, $f(x_1, x_2) = 1$ when $x_1 = 0$, and $+\infty$ elsewhere.) The domain is stored as a planar arrangement data structure provided by the computational geometry library CGLAB [10]. (Internally it is stored as a list of vertices, edges, and faces with pointers

to adjacent entities while each entity E_i has a pointer to a data structure storing A_i , b_i , and c_i .)

More precisely, a PLQ function is defined as (v_0, e_0, f_0, F) where v_0 is a $3 \times n_v$ matrix (where n_v is the number of vertices in the arrangement) storing each vertex as a column vector in v_0 in the format $[x; y; i]$ where (x, y) are the coordinates of the vertex and i is the index of a column in F . The edges are stored in e_0 , a $6 \times n_e$ matrix (where n_e is the number of edges in the arrangement). Each column of e_0 stores one edge in the format $[type; v_i; 0; v_j; 0; i]$ (resp. $[type; v_i; 0; x_j; y_j; i]$, $[type; x_i; y_i; x_j; y_j; i]$) where type is 0 (resp. 1, 2) for a segment (resp. ray, line) and v_i (resp. v_j) is a column index in v_0 defining the start of the segment or ray. The values (x_i, y_i) (resp. (x_j, y_j)) define a point on the line or ray. The integer i is the index of a column in F . The faces are defined in f_0 as column vectors in the format $[v_i, e_i, lr, i]$ where v_i is the index of a column in v_0 defining a vertex, e_i is the index of a column in e_0 defining an edge, lr is 0 (resp. 1) for the face which is counterclockwise (resp. clockwise) when one looks at the ray with vertex $v_0(v_i, :)$ and edge $e_0(e_i, :)$. The integer i is the index of a column in F . Finally, F stores the quadratic functions associated with each entity. Each quadratic function is a column in F in the format $[a; b; c; d; e; g]$ where $f(x, y) = ax^2 + bxy + cy^2 + dx + ey + g$.

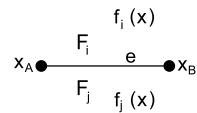
For example, $f(x_1, x_2) = |x_1| + |x_2|$ is represented as

$$v_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad e_0 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & 2 & 3 & 4 \end{bmatrix},$$

$$f_0 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 0 & 0 & 0 & 0 \\ 1 & 2 & 3 & 4 \end{bmatrix}, \quad F = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

From v_0 we deduce there is only one vertex at $(0, 0)$ with value given by $F(:, 1)$. There are four rays since the first row of e_0 is $[1, 1, 1, 1]$. All four start from the lone vertex with index $v_i = 1$ (second row of e_0 is $[1, 1, 1, 1]$), the third row of e_0 is not used and filled with zeros, the fourth and fifth rows give the coordinates of a point that indicates the direction of the ray e.g. $(1, 0)$ for the first ray. The sixth row indicates the index of the column in F . For example, the second ray in e_0 is $[1; 1; 0; 0; 1; 2]$ meaning it starts at $(0, 0)$, points to $(0, 1)$ and has parameters $a = 0$, $b = 0$, $c = 0$, $d = -1$, $e = 1$, $g = 0$ so the function is $-x_1 + x_2$. As for the faces, f_0 indicates there are four faces (four columns). The first face is coded as $[1; 1; 0; 1]$ to indicate it is the face counterclockwise to the ray starting at $(0, 0)$, and pointing toward $(1, 0)$. Its associated function is coded as the first column in F , i.e. $|x_1| + |x_2|$.

We will call a QQ (resp. QL, LL) face a face associated with a quadratic function with a symmetric positive definite (resp. positive semi-definite, zero) matrix. In other

Fig. 1 An edge to conjugate

words, faces are classified by the number of nonzero eigenvalues of their associated quadratic function.

To compute $f(x)$ at $x \in \mathbb{R}^2$ we find the entity E_i to which x belongs using a planar point-location algorithm [10, Sect. 30.3.1], then we evaluate the associated quadratic function f_i at x . Consequently, computing $f(x)$ where f is a bivariate PLQ function containing $O(n)$ entities takes $O(\log n)$ time in the worst-case.

The addition of two bivariate PLQ functions is a bivariate PLQ function that can be computed using a map overlay algorithm [14, Chap. 2], and adding the quadratic functions associated with the resulting overlay.

The scalar multiplication operation is even simpler as the planar arrangement is unmodified; only the quadratic functions are multiplied. Similarly, adding a quadratic function to a bivariate PLQ function only requires to add the quadratic function to each of functions f_i associated with an entity E_i .

The core computation of the conjugate relies on the following well-known fact in convex analysis.

Fact 2.3 ([23, XI Proposition 1.2.1] and [41, Proposition 11.3]) *Assume $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a proper, lsc, convex function, and let $x, s \in \mathbb{R}^n$. Then*

$$f^*(s) = \langle s, x \rangle - f(x) \Leftrightarrow s \in \partial f(x) \Leftrightarrow x \in \partial f^*(s).$$

A convex bivariate PLQ function has a special structure that induces a primal and a dual partition. First, we note that by [41, Proposition 11.3] $\text{dom } \partial f^* = \text{rge } \partial f$. Consequently, $\text{dom } \partial f^* = \bigcup_i \partial f(P_i)$. The fundamental idea of the algorithm is to loop through the primal entities and compute their associated dual entities.

We now classify edges according to the quadratic forms associated to their neighborhood faces. An edge is called a *linear edge* when the restriction of the function f to the edge is linear. Furthermore, a linear edge is *smooth* (resp. *kinked*) if the gradient of the neighborhood quadratic function are equal (resp. not equal) at one point on the edge (in that case the gradient is constant on the edge): $\nabla f_i(x_A) = \nabla f_j(x_A)$ (resp. $\nabla f_i(x_A) \neq \nabla f_j(x_A)$). See Fig. 1.

Edges on which the function f is a quadratic nonlinear function are called *quadratic edges*. Similarly, we distinguish *smooth* quadratic edges (when the function f is continuously differentiable on the relative interior of the union of the two neighbor faces) from *kinked* quadratic edges (when f is not continuously differentiable on the relative interior of the union of the two neighbor faces).

Note that there is no restriction between the type of a primal entity and the type of its conjugate: primal vertices may be associated with dual faces, primal faces may be associated with dual edges, etc. The entity associated with a primal vertex is determined by the value of the gradients of the faces incident to the vertex and the normal cone of the effective domain at the vertex. The dual entity associated with primal

faces conjugate depend on the positive definiteness of the A_i matrix assigned to their function value.

3 The partial conjugate

Assume $f : \mathbb{R}^{m+n} \rightarrow \mathbb{R} \cup \{+\infty\}$ is a proper convex PLQ function of $m+n$ variables. The partial conjugate of f , denoted f^{*1} (resp. f^{*2}), is defined as the conjugate of f with respect to the first m (resp. last n) variables *i.e.*

$$f^{*1}(s, x) = \sup_y [s^T y - f(y, x)] \quad \left(\text{resp. } f^{*2}(y, t) = \sup_x [t^T x - f(y, x)] \right).$$

For a bivariate function f , we arbitrarily look at taking the partial conjugate with respect to x_1 , which becomes

$$f^{*1}(s_1, x_2) = \sup_{x_1 \in \mathbb{R}} [s_1 x_1 - f(x_1, x_2)].$$

We will use the same notation as in [6, 38] for the switching operator

$$f^\top(x_1, x_2) = f(x_2, x_1).$$

Computing the (full) conjugate of a bivariate function amounts to two partial conjugate computations *i.e.* $f^* = (-f^{*1})^{*2}$, see [40, Theorems 33.3 and 34.2]. We also only need to implement one partial conjugate as we point out next (we omit the straightforward proof).

Fact 3.1 *The following formula holds*

$$f^* = ([-(f^{*1})^\top]^{*1})^\top. \quad (2)$$

Equation (2) is useful in the implementation of a partial conjugate algorithm, as it requires only considering the partial conjugate with respect to a single fixed variable. Arbitrarily, we choose the first variable.

To adapt the full conjugate algorithm [19] to the partial conjugate, we recall the following fact. (The notation $\partial_1 f$ refers to the subgradient of the function $x_1 \mapsto f(x_1, x_2)$ *i.e.* the partial subgradient with respect to the first variable.) Since we are working with negatives of extended real numbers, we now have $f^{*1} : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty, -\infty\}$. (In computing the conjugate, we will make sure we do not use any undefined operations such as ∞/∞ , $\infty - \infty$, or $0 \cdot \infty$.)

We assume f is convex so the function f^{*1} is convex-concave by [41, Proposition 11.48].

Remark 3.2 While it is clear that the function f^{*1} is separately PLQ, one cannot immediately deduce it is jointly PLQ since there are separately PLQ functions that are not jointly PLQ *e.g.*, $f(x_1, x_2) = \max(1, x_1^2 + x_2^2)$. We cannot apply [41, Proposition 11.32] since the term $-s_1 x_1$ is not convex. Similarly, assuming (or proving) that

the supremum is attained does not allow one to use [41, Exercise 10.22 (c)] since the point $\bar{x}_1$ where the supremum is attained depends on (s_1, x_2) . Fact 3.3 below shows that the partial conjugate is indeed a PLQ function.

The next result, due to Goebel, ensures that partial conjugates are still PLQ.

Fact 3.3 (Partial conjugates are PLQ [20, Proposition 3, p. 17]) *Assume $f : \mathbb{R}^{m+n} \rightarrow \mathbb{R} \cup \{+\infty\}$ is a proper convex PLQ function of $m+n$ variables. Let $f^{\star 1}$ be the partial conjugate of f with respect to the first m variables. Then $f^{\star 1}$ is PLQ.*

We now provide a constructive proof showing that the partial conjugate of a convex PLQ bivariate function is PLQ. First, we need a couple of lemmas.

Fact 3.4 [41, Corollary 11.16] *Assume f is a proper PLQ bivariate function. If $f^{\star 1}(s_1, x_2)$ is finite for some $(s_1, x_2) \in \mathbb{R}^2$, then the supremum is attained in the definition of $f^{\star 1}$ i.e. $\forall (s_1, x_2) \in \text{dom } f^{\star 1}, \exists x_1 \in \mathbb{R}, f^{\star 1}(s_1, x_2) = s_1 x_1 - f(x_1, x_2)$.*

Lemma 3.5 *Assume f is proper convex PLQ with $\text{dom } f = \bigcup_i P_i$. Then $\text{dom } f^{\star 1} = \bigcup_i \partial_{10} f(P_i)$ where*

$$\partial_{10} f(x_1, x_2) = \partial_1 f(x_1, x_2) \times \{x_2\}.$$

Proof The result follows from the following equivalences.

$$\begin{aligned} y \in \partial_{10} f(x_1, x_2) &\Leftrightarrow \exists i, \exists x \in P_i, y_1 \in \partial_1 f(x) \text{ and } y_2 = x_2, \\ &\Leftrightarrow \exists i, \exists x \in P_i, f^{\star 1}(y_1, x_2) = y_1 x_1 - f(x) \text{ and } y_2 = x_2, \\ &\Leftrightarrow \exists i, \exists x \in P_i, (y_1, x_2) \in \text{dom } f^{\star 1} \text{ and } y_2 = x_2, \\ &\Leftrightarrow y \in \text{dom } f^{\star 1}. \end{aligned}$$

The third equivalence holds because f is convex in x_1 , and the fourth by Lemma 3.4. $\square$

Now we restate Fact 3.3 for bivariate functions and include a constructive proof that follows closely the algorithm we use to compute the partial conjugate.

Theorem 3.6 *Assume $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ is a proper convex PLQ bivariate function. Then $f^{\star 1}$ is PLQ.*

Proof We note $\text{dom } f = \bigcup_{i=1}^r P_i$ where P_i is a polyhedral set on which $f = f_i$ is a quadratic function.

When $\dim \text{dom } f = 0$ since f is convex the domain is a singleton: $\text{dom } f = \{(\bar{x}_1, \bar{x}_2)\}$. So when $x_2 = \bar{x}_2$, $f^{\star 1}(s_1, x_2) = s_1 \bar{x}_1 - f(\bar{x}_1, \bar{x}_2)$ is affine. Hence $f^{\star 1}$ is PLQ.

When $\dim \text{dom } f = 1$ we consider the line $\text{aff dom } f$. If $\text{aff dom } f$ is a horizontal line, for all $(x_1, x_2) \in \text{dom } f$, $x_2 = \bar{x}_2$ is constant and $\underline{x}_1 \leq x_1 \leq \bar{x}_1$ so $f(x_1, x_2) =$

$g(x_1)$ only depends on x_1 and $f^{\star 1}(s_1, x_2) = g^{\star}(s_1)$. Then $f^{\star 1}$ is PLQ as the (full) conjugate of a (univariate) PLQ function by [41, Exercise 10.22, Example 11.28, Proposition 11.32]. Otherwise, $\text{aff dom } f$ is a nonhorizontal line *i.e.* $\text{aff dom } f = \{(x_1, x_2) : x_1 = \alpha x_2 + \beta\}$ where $\alpha, \beta \in \mathbb{R}$. Then $f^{\star 1}(s_1, x_2) = s_1(\alpha x_2 + \beta) - f(\alpha x_2 + \beta, x_2)$ is PLQ as the composition of a PLQ function with a linear function, see [41, Exercise 10.22(b)]. So $f^{\star 1}$ is PLQ as the conjugate of a univariate PLQ function.

The only remaining case is when $\text{int dom } f$ is nonempty. By [41, Lemma 2.50] we can always assume $\text{int } P_i$ is nonempty for all i , and $\text{int } P_i \cap \text{int } P_j$ is empty for all $i \neq j$. Using Lemma 3.5 we have

$$\text{dom } f^{\star 1} = \bigcup_{i=1}^r \partial_{10} f(P_i).$$

Any P_i can be written as the union of its relative interior and its edges: $P_i = \text{ri } P_i \cup \bigcup_j E_j^i$ where E_j^i is an edge in P_i *i.e.* $\text{bd } P_i = \bigcup_j E_j^i$ (the notation bd denotes the boundary). Then

$$\partial_{10} f(P_i) = \partial_{10} f(\text{ri } P_i) \cup \bigcup_j \partial_{10} f(E_j^i).$$

Each set on the right-end side is a polyhedral set on which f is PLQ by Lemmas 3.7–3.13. All that remains to be shown is that all cases are captured by Lemmas 3.7–3.13 below.

Lemma 3.7 covers the case of horizontal edges. For nonhorizontal edges, the cases are split depending on whether the functions on each side of the edge are finite or always equal to $+\infty$. Lemma 3.8 takes care of the case when the function f is finite and quadratic nonlinear on the edge and equals $+\infty$ everywhere else (by convexity this also falls in the $\dim \text{dom } f = 1$ case above). Lemma 3.9 handles the case when only one adjacent face is associated with the $+\infty$ function. The remaining case of having an edge surrounded by finite quadratic functions is split into three subcases depending on whether the edge is a segment (Lemma 3.10), ray (Lemma 3.11), or line (Lemma 3.12).

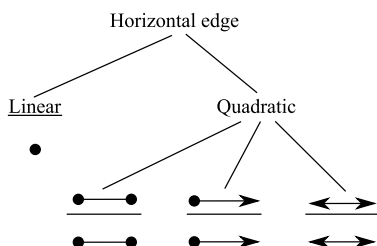
Finally, Lemma 3.13 handles the case when the entity is a (full dimensional) face.

Hence, all cases have been considered and the proof is complete. $\square$

Lemmas 3.7–3.13 enumerate all the possible cases and ensure the correctness of our partial conjugate algorithm. First, we deal with the entities dual to a primal edge. We note E an edge

$$E = \{(x_1, x_2) : \alpha x_1 + \beta x_2 = \gamma, \underline{x}_1 \leq x_1 \leq \overline{x}_1, \underline{x}_2 \leq x_2 \leq \overline{x}_2\}$$

where $\alpha + \beta = 1$, $\underline{x}_1, \underline{x}_2 \in \mathbb{R} \cup \{-\infty\}$ and $\overline{x}_1, \overline{x}_2 \in \mathbb{R} \cup \{+\infty\}$. A *horizontal* edge corresponds to $\alpha = 0$. An edge is a *line* when $\underline{x}_1 = \underline{x}_2 = -\infty$ and $\overline{x}_1 = \overline{x}_2 = +\infty$. It is a *ray* if either $\underline{x}_1 = \underline{x}_2 = -\infty$ or $\overline{x}_1 = \overline{x}_2 = +\infty$ but not both. We call an edge a *segment* when $\underline{x}_1, \underline{x}_2, \overline{x}_1, \overline{x}_2$ are all finite. In that case, $E = [\underline{x}, \overline{x}]$ where we use the notation $\underline{x} = (\underline{x}_1, \underline{x}_2)$ and $\overline{x} = (\overline{x}_1, \overline{x}_2)$.

Fig. 2 Edge partial conjugate cases (horizontal edges)

Lemma 3.7 (Horizontal edges) Assume E is a horizontal edge. Then its dual entity $\partial_{10}f(E)$ is given as follow.

1. if $f|_E$ (the restriction of f to E) is linear then $\partial_{10}f(E)$ is a singleton.
2. Otherwise $f|_E$ is a quadratic nonlinear function. If E is a (horizontal) segment (resp. ray, line) then $\partial_{10}f(E)$ is a horizontal segment (resp. ray, line).

In all cases, f^* is PLQ on $\partial_{10}f(E)$.

Proof We note $E = \{(x_1, \gamma) : \underline{x}_1 \leq x_1 \leq \overline{x}_1\}$ and for any $x \in E$,

$$f(x_1, x_2) = a(x_1)^2 + bx_1x_2 + c(x_2)^2 + dx_1 + ex_2 + g.$$

Take $(s_1, y_2) \in \partial_{10}f(E)$ i.e. there is $x \in E$ with $s_1 \in \partial_1 f(x)$ and $y_2 = x_2$. When $f|_E$ is linear we have $a = 0$. So this is further equivalent to the existence of x with $\underline{x}_1 \leq x_1 \leq \overline{x}_1$, $x_2 = \gamma$, $s_1 = bx_2 + d$, and $y_2 = x_2$. In other words, $s_1 = b\gamma + d$ and $y_2 = \gamma$. Hence, $\partial_{10}f(E) = \{(b\gamma + d, \gamma)\}$ is a singleton.

Now assume $f|_E$ is quadratic nonlinear. The inclusion $(s_1, y_2) \in \partial_{10}f(E)$ means there is $x_1 \in [\underline{x}_1, \overline{x}_1]$ such that $s_1 = 2ax_1 + b\gamma + d$ and $y_2 = \gamma$. In other words,

$$\partial_{10}f(E) = 2a[\underline{x}_1, \overline{x}_1] + b\gamma + d \times \{\gamma\}.$$

The fact that f is PLQ on $\partial_{10}f(E)$ follows. $\square$

The decision tree drawn in Fig. 2 illustrates Lemma 3.7.

To tackle nonhorizontal edges, we need to know whether f is finite on the neighboring faces.

Lemma 3.8 (Nonhorizontal edge ∞/∞) Assume $E = F_1 \cap F_2$ where F_i are faces (polyhedral set with nonempty interior) in the decomposition of $\text{dom } f = \bigcup_i P_i$ and $f|_{F_i} \equiv +\infty$ for $i = 1, 2$.

If E is a segment (resp. ray, line) then $\partial_{10}f(E) = \mathbb{R} \times [\underline{x}_2, \overline{x}_2]$ on which f^* is PLQ.

Proof Assume $E = \{(x_1, x_2) : x_1 = \beta x_2 + \gamma, \underline{x}_2 \leq x_2 \leq \overline{x}_2\}$. For any x_2 there is a unique $x_1 = \beta x_2 + \gamma$ at which f is finite. In fact,

$$f(x_1, x_2) = f(x_1, x_2) + \iota_{\{\beta x_2 + \gamma\}}(x_1).$$

A univariate conjugate computation gives $\partial_1 f(x_1, x_2) = \mathbb{R}$ when $\underline{x_2} \leq x_2 \leq \overline{x_2}$. In that case we have $f^{*1}(s_1, x_2) = s_1(\beta x_2 + \gamma) - f(\beta x_2 + \gamma, x_2)$. Otherwise $\partial_1 f(x_1, x_2) = \emptyset$ and $f^{*1}(s_1, x_2) = +\infty$. In both cases, f^{*1} is PLQ. $\square$

Lemma 3.9 (Nonhorizontal edge f_i/∞) *Assume $E = F_1 \cap F_2$ where F_i are faces in the decomposition of $\text{dom } f = \bigcup_i P_i$ and*

$$f|_{\text{ri } F_1}(x_1, x_2) = f_1(x_1, x_2) = a(x_1)^2 + bx_1x_2 + c(x_2)^2 + dx_1 + ex_2 + g,$$

while $f|_{\text{ri } F_2} \equiv +\infty$. Set $H = \{x : x_1 \geq \beta x_2 + \gamma\}$ and assume $F_2 \subset H$.

Assume $E = \{x : x_1 = \beta x_2 + \gamma, \underline{x_2} \leq x_2 \leq \overline{x_2}\}$ is a nonhorizontal edge. Set $H^* = \{(s_1, x_2) : s_1 \geq 2a(\beta x_2 + \gamma) + bx_2 + d\}$, $H_- = \{x : \underline{x_2} \leq x_2\}$, and $H_+ = \{x : x_2 \leq \overline{x_2}\}$. Then $\partial_{10} f(E) = H^* \cap H_- \cap H_+$ on which f^{*1} is PLQ.

Proof For any x_2 with $\underline{x_2} \leq x_2 \leq \overline{x_2}$ and $x_1 \in F_1 \cup F_2$, we have $f(x_1, x_2) = f_1(x_1, x_2) + \iota_H(x_1)$. Hence, for any $s_1 \geq \frac{\partial f_1}{\partial x_1}(x)$ and $x_1 = \beta x_2 + \gamma$ we have $f^{*1}(s_1, x_2) = s_1 x_1 - f_1(x)$ and $\partial_1 f(x) = [\frac{\partial f_1}{\partial x_1}(x), +\infty[$. The result follows from $\frac{\partial f_1}{\partial x_1}(\beta x_2 + \gamma, x_2) = 2ax_1 + bx_2 + d$. $\square$

When the edge forms the boundary of two faces on which f is finite, we need to introduce the following notation. For a quadratic function f_1 , we name

$$\nabla_{10} f(x_1, x_2) = \left[\frac{\partial f}{\partial x_1}(x_1, x_2) \right]_{x_2}.$$

Moreover, given a point x , we define the boolean function

$$A(x) = A(x; f_1, f_2) = \begin{cases} \text{true} & \text{if } \nabla_{10} f_1(x_1, x_2) = \nabla_{10} f_2(x_1, x_2), \\ \text{false} & \text{otherwise.} \end{cases}$$

Lemma 3.10 (Nonhorizontal edge f_i/f_j segment) *Assume $E = F_1 \cap F_2 = [\underline{x}, \overline{x}]$ is a segment where F_i are faces in the decomposition of $\text{dom } f = \bigcup_i P_i$, and $f|_{F_1} = f_1$ (resp. $f|_{F_2} = f_2$) is a quadratic function.*

1. If $A(\underline{x})$ and $A(\overline{x})$ hold, $\partial_{10} f(E)$ is a segment.
2. If $A(\overline{x})$ hold but not $A(\underline{x})$, $\partial_{10} f(E)$ is a triangle with one horizontal side.
3. If $A(\underline{x})$ hold but not $A(\overline{x})$, $\partial_{10} f(E)$ is a triangle with one horizontal side.
4. If neither $A(\underline{x})$ nor $A(\overline{x})$ hold, $\partial_{10} f(E)$ is a polyhedral set with four edges, two of them being horizontal edges.

In all cases, f^{*1} is PLQ on $\partial_{10} f(E)$.

Proof Consider Case (i). We have $\frac{\partial f_1}{\partial x_1}(\underline{x}) = \frac{\partial f_2}{\partial x_1}(\underline{x})$ and $\frac{\partial f_1}{\partial x_1}(\overline{x}) = \frac{\partial f_2}{\partial x_1}(\overline{x})$. Because f_1 and f_2 are quadratic, their first derivatives are equal on all E . So the function f is convex continuously differentiable on $\text{ri}(F_1 \cup F_2) \cap \mathbb{R} \times [\underline{x_2}, \overline{x_2}]$. Consequently,

$$\partial_{10} f(E) = \left[\left(\frac{\partial f_1}{\partial x_1}(\underline{x}), \underline{x_2} \right), \left(\frac{\partial f_1}{\partial x_1}(\overline{x}), \overline{x_2} \right) \right].$$

and when $\underline{x}_2 \leq x_2 \leq \overline{x}_2$ we have $x_1 = \beta x_2 + \gamma$, $s_1 = \frac{\partial f_1}{\partial x_1}(x)$ and $f^{\star 1}(s_1, x_2) = s_1 x_1 - f_1(x)$ which proves (i) and the fact $f^{\star 1}$ is PLQ in that case.

In Case (ii), $\frac{\partial f_1}{\partial x_1}(\underline{x}) \neq \frac{\partial f_2}{\partial x_1}(\underline{x})$ and $\frac{\partial f_1}{\partial x_1}(\overline{x}) = \frac{\partial f_2}{\partial x_1}(\overline{x})$. So $\partial_{10} f(x) = [\frac{\partial f_1}{\partial x_1}(x), \frac{\partial f_2}{\partial x_1}(x)] \times \{x_2\}$. For any $\underline{x}_2 \leq x_2 \leq \overline{x}_2$ and $s_1 \in [\frac{\partial f_1}{\partial x_1}(x), \frac{\partial f_2}{\partial x_1}(x)]$, $x_1 = \beta x_2 + \gamma$ and $f^{\star 1}(s_1, x_2) = s_1 x_1 - f(x)$. Consequently, $\partial_{10} f(E) = \text{co}\{\nabla_{10} f_1(\underline{x}), \nabla_{10} f_2(\underline{x}), \nabla_{10} f_1(\overline{x})\}$, and the result follows.

Case (iii) is similar to Case (ii) while Case (iv) is a simple adaptation of Case (ii). $\square$

Lemma 3.11 (Nonhorizontal edge f_i/f_j ray) *Assume $E = F_1 \cap F_2$ is a ray where F_i are faces in the decomposition of $\text{dom } f = \bigcup_i P_i$, and $f|_{F_1} = f_1$ (resp. $f|_{F_2} = f_2$) is a quadratic function. Name $\underline{x}$ such that $E = \{x : x_1 = \beta x_2 + \gamma, x_2 \geq \underline{x}_2\}$ i.e. $\underline{x}$ is the base point of the ray. Take $\overline{x} \in E$ with $\overline{x} \neq \underline{x}$.*

1. If $A(\underline{x})$ and $A(\overline{x})$ hold, $\partial_{10} f(E)$ is a ray.
2. If $A(\overline{x})$ hold but not $A(\underline{x})$, the function f is not convex.
3. If $A(\underline{x})$ hold but not $A(\overline{x})$, $\partial_{10} f(E)$ is a face bounded by two nonhorizontal rays.
4. If neither $A(\underline{x})$ nor $A(\overline{x})$ hold, $\partial_{10} f(E)$ is a face whose boundary is made of an horizontal segment and two nonhorizontal rays.

In cases (i), (iii), and (iv), $f^{\star 1}$ is PLQ on $\partial_{10} f(E)$.

Proof Similar to the proof of the previous lemma except for (ii) which is [19, Lemma 1]. $\square$

Lemma 3.12 (Nonhorizontal edge f_i/f_j line) *Assume $E = F_1 \cap F_2 = \{x : x_1 = \beta x_2 + \gamma\}$ is a line where F_i are faces in the decomposition of $\text{dom } f = \bigcup_i P_i$, and $f|_{F_1} = f_1$ (resp. $f|_{F_2} = f_2$) is a quadratic function. Name $\underline{x}$ and $\overline{x}$ two distinct points in E .*

1. If $A(\underline{x})$ and $A(\overline{x})$ hold, f is continuously differentiable on $\text{ri } F_1 \cap F_2$, and $\partial_{10} f(E)$ is a line. (We call E a smooth edge in that case.)
2. Otherwise neither $A(\underline{x})$ nor $A(\overline{x})$ hold, $\partial_{10} f(E)$ is a face bounded by two lines. (We call E a kinked edge in that case.)

In all cases, $f^{\star 1}$ is PLQ on $\partial_{10} f(E)$.

Proof Similar to the proof of the previous lemma. [19, Lemma 1] shows that there are only two cases to consider. $\square$

Lemma 3.13 (Face) *Assume E is a face in the decomposition of $\text{dom } f = \bigcup_i P_i$. Then $f^{\star 1}$ is PLQ on $\partial_{10} f(E)$, which is a polyhedral set.*

Proof First, note that

$$\begin{aligned} (s_1, x_2) \in \partial_{10} f(\text{int } E) &\Leftrightarrow \exists y \in \text{int } E (s_1, x_2) \in \partial_{10} f(y), \\ &\Leftrightarrow \exists y \in \text{int } E s_1 \in \partial_1 f(y) \text{ and } x_2 = y_2, \end{aligned}$$

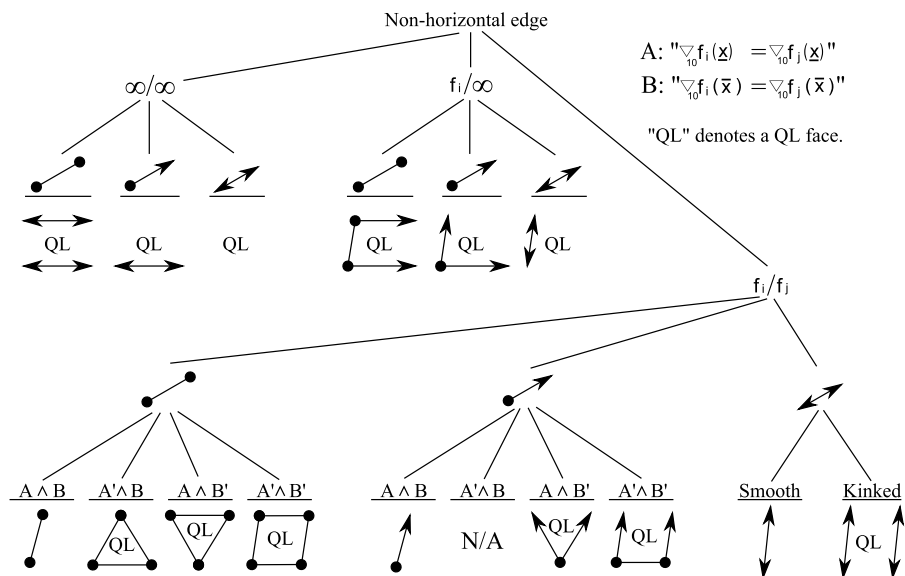


Fig. 3 Edge partial conjugate cases (non-horizontal edges)

$$\Leftrightarrow \exists y \in \text{int} E s_1 = 2ay_1 + by_2 + d \text{ and } x_2 = y_2,$$

$$\Leftrightarrow (s_1, x_2) \in L(\text{int} E),$$

where we write

$$f|_{\text{int} E}(x_1, x_2) = a(x_1)^2 + bx_1x_2 + c(x_2)^2 + dx_1 + ex_2 + g,$$

and L is the linear application defined by $L(y) = (2ay_1 + by_2 + d, y_2)$. We deduce $\text{cl } \partial_{10} f(\text{int } E) = \text{cl } L(\text{int } E)$ is polyhedral. Now take $(s_1, x_2) \in \partial_{10} f(\text{int } E)$. If $a = 0$ we have

$$f^{*1}(s_1, x_2) = -c(x_2)^2 - ex_2 - g + \iota_{\{bx_2+d\}}(s_1).$$

Otherwise $a > 0$ and $f^{*1}(s_1, x_2) = s_1x_1 - f(x)$ where $x_1 = \frac{s_1 - bx_2 - d}{2a}$. In both cases, f^{*1} is PLQ on $\partial_{10} f(\text{int } E)$ and since it is continuous on E (as a convex-concave function), it is PLQ on $\partial_{10} f(E)$. $\square$

We summarize the nonhorizontal edge cases in the decision tree shown in Fig. 3. The linear space complexity of the computation of the partial Moreau envelope

$$e_\lambda^1(s_1, x_2) = \inf_{x_1} \left[f(x_1, x_2) + \frac{(x_1 - s_1)^2}{2\lambda} \right],$$

the partial conjugate, and the conjugate of a bivariate convex PLQ function follow immediately from our algorithm. This fact was noted experimentally in [35] for the Moreau envelope.

Corollary 3.14 *Computing the partial conjugate, the partial Moreau envelope, the full conjugate, and the full Moreau envelope of a bivariate proper convex PLQ function can be done in linear worst-case space complexity.*

Proof Each primal entity is associated with $O(1)$ dual entities, hence the partial conjugate algorithm has linear space complexity. The result extends to the full conjugate using Formula (2) and to the partial and full Moreau envelope using Formula (1). $\square$

4 Partial conjugate algorithm

Before discussing conjugate entities, we present some examples for which we have computed the partial conjugate using our algorithm.

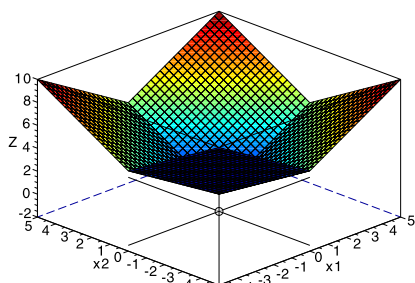
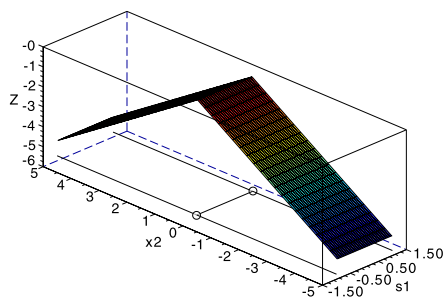
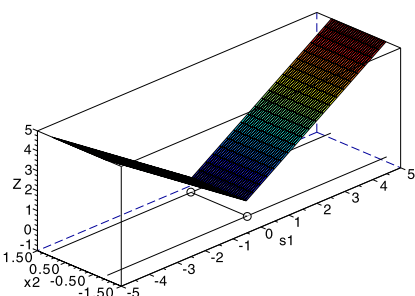
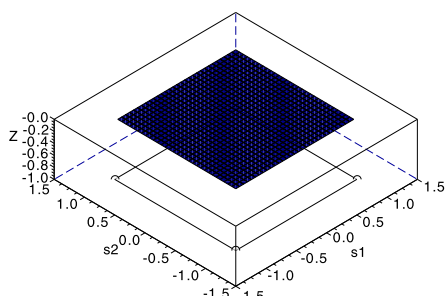
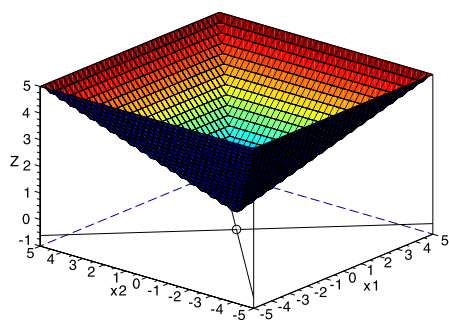
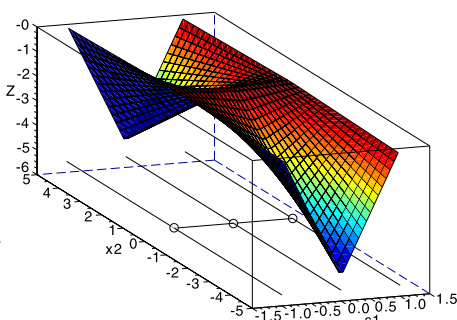
The algorithm is implemented as a Scilab [11] toolbox. The Computational Geometry Algorithms Library CGAL [9], an open source C++ package with a variety of numerically stable geometric data structures and algorithms, was used to store and manipulate arrangements of PLQ2d function domains. We used the Scilab toolbox CGLAB [10] to interface with CGAL, and extended it to have access to planar subdivisions. Each conjugate example image in this paper was generated from either the conjugate algorithm in [18], or our new partial conjugate algorithm.

Example 4.1 Consider the ℓ_1 norm, $f(x_1, x_2) = |x_1| + |x_2|$, with partial conjugate $f^{*1}(s_1, x_2) = -|x_2| + \iota_{[-1,1]}(s_1)$. Figures 4(a) and 4(b) show these functions, respectively. We recover the full conjugate by further computing $-(f^{*1})^\top$ and f^* (from Eq. (2)) in Figs. 4(c) and 4(d).

Example 4.2 The ℓ_∞ norm, shown in Fig. 5(a) is defined as $f(x_1, x_2) = \max\{|x_1|, |x_2|\}$. Its partial conjugate can be seen in Fig. 5(b). The function value on the finite piece in the positive quadrant of f^{*1} is $s_1 x_2 - x_2$ (the other function values are symmetric). The input function is piecewise linear, whereas the faces of its partial conjugate have four indefinite quadratic forms. This is in contrast to the full conjugate of a piecewise linear function, which is always piecewise linear [41, Theorem 11.14]. The four faces in the input function are not associated with the faces in the partial conjugate (it is the primal edges, not the faces, that are associated with the dual faces).

Example 4.3 We next demonstrate the partial conjugate of the indicator function of a convex polygon. The function is $f(x_1, x_2) = \iota_C(x_1, x_2)$, where $C = \text{co}\{(-1, -1), (1, -1), (-1, 1), (1, 1), (0, 2)\}$. Figure 6(a) shows the function. The partial conjugate, shown in Fig. 6(b), is unbounded in s_1 and bounded in x_2 , convex in s_1 and concave in x_2 (when the other variable is fixed).

Example 4.4 The function $f(x_1, x_2) = x_2^2 + \iota_{[-1,1]}(x_1)$ is linear in x_1 and quadratic in x_2 . As this function is separable, its partial conjugate is easy to compute as $f^{*1}(s_1, x_2) = |s_1| - x_2^2$. Figures 7(a) and 7(b) demonstrate the behavior of the partial conjugate on this function.

(a) The ℓ_1 norm, $f(x_1, x_2) = |x_1| + |x_2|$.(b) The partial conjugate $f^*(s_1, x_2)$.(c) Switching variables and negating gives $g = -(f^*)^T$.(d) The conjugate is recovered from $f^* = (g^*)^T$.**Fig. 4** Computing the conjugate of the ℓ_1 norm(a) The ℓ_∞ norm, $f(x_1, x_2) = \max\{|x_1|, |x_2|\}$.(b) The partial conjugate of the ℓ_∞ norm.**Fig. 5** The ℓ_∞ norm and its partial conjugate

The partial conjugate leads naturally to the partial Moreau envelope and the partial proximal average

$$\mathcal{P}_\mu^1(f_0, f_1; \lambda_0, \lambda_1)(x) = e_\mu^1(-\lambda_0 e_\mu^1 f_0 - \lambda_1 e_\mu^1 f_1).$$

We present two examples to illustrate these operators.

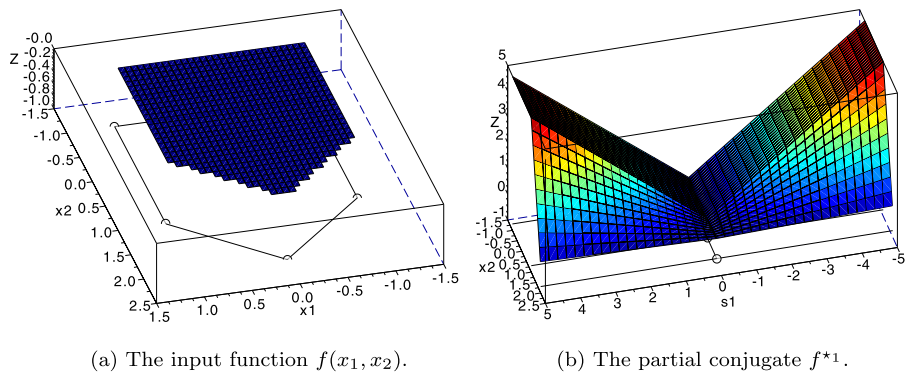


Fig. 6 The partial conjugate of the indicator of a convex polygon

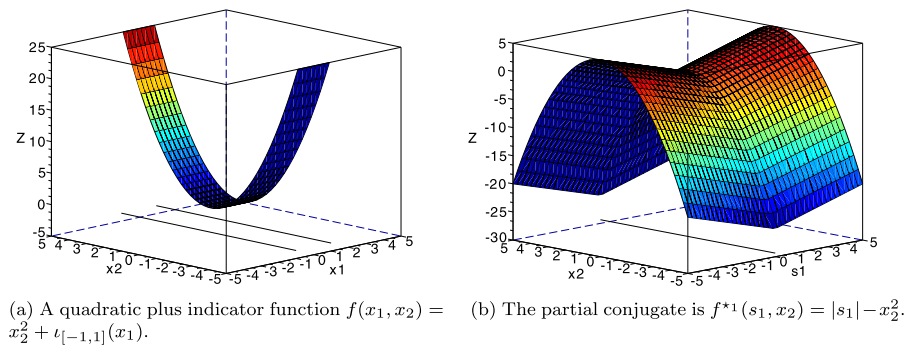


Fig. 7 The partial conjugate of a function that is linear in x_1 and quadratic in x_2

Example 4.5 Consider the piecewise function f defined by

$$f(x_1, x_2) = \begin{cases} x_1^2 + x_2^2, & \text{if } x_1 \geq 0, x_2 \geq 0; \\ -2x_1 + x_2^2, & \text{if } x_1 < 0, x_2 \geq 0; \\ x_1^2 - 2x_2, & \text{if } x_1 \geq 0, x_2 < 0; \\ -2x_1 - 2x_2, & \text{if } x_1 < 0, x_2 < 0. \end{cases}$$

This function and its partial conjugate are shown, respectively, in Figs. 8(a) and 8(b). The partial Moreau envelope $e_1^1 f$ is shown in Fig. 8(c). It is full-domain, and smooth in s_1 for any fixed x_2 but non-smooth in x_2 for any fixed s_1 .

Example 4.6 We consider the proximal average of the ℓ_1 and ℓ_∞ norms for different values of λ_2 (with $\lambda_1 + \lambda_2 = 0$). When $\lambda_2 = 0$, the partial proximal average $\mathcal{P}_1^1(\|\cdot\|_1, \|\cdot\|_\infty; \lambda_1, \lambda_2)$ is equal to $\|\cdot\|_1$, and correspondingly is equal to $\|\cdot\|_\infty$ when $\lambda_2 = 1$. Figures 9(a) and 9(b) show this partial proximal average for different values of λ_2 . For both $\lambda_2 = 0.25$ and $\lambda_2 = 0.5$, the domain is comprised of 14 different faces.

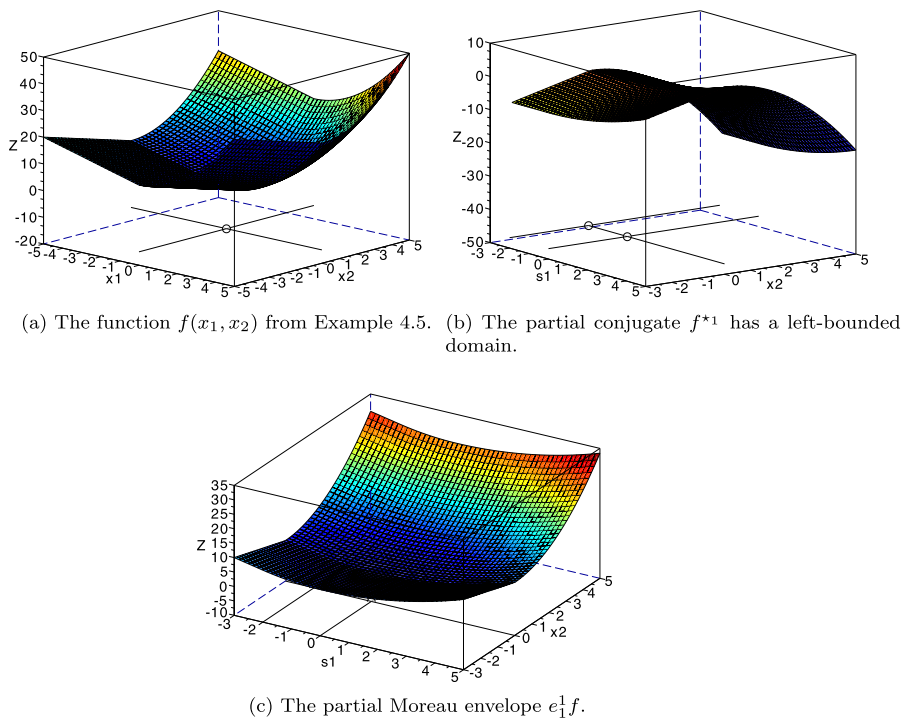


Fig. 8 The partial conjugate and partial Moreau envelope of a full-domain function

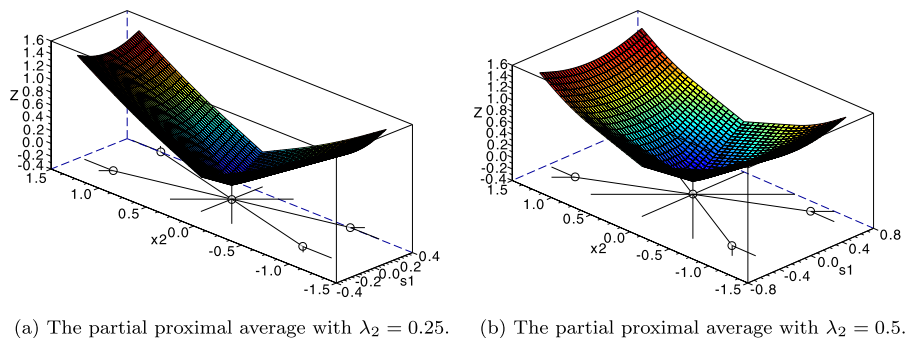


Fig. 9 The partial proximal average of $\|\cdot\|_1$ and $\|\cdot\|_\infty$ for different values of (λ_1, λ_2)

With these examples in mind, we follow the path of the full conjugate algorithm and seek the dual conjugate entities associated with each primal entity in the domain of the input function. Algorithm 1 shows the structure of the algorithm. The special cases are described in Sect. 4.3.

The worst-case time complexity of the algorithm is the same as the full conjugate algorithm. The key step is merging the dual entities into the dual arrangement, which

Algorithm 1: PLQ2d convex partial conjugate

Input: f (convex PLQ2d function)
Output: $f^{\star 1}$ (convex-concave PLQ2d function, the partial conjugate of f)

```

1 begin
2   Check for and handle special cases ;
3   foreach Primal Edge and Primal Face do
4     Construct the conjugate entity ;
5     Merge it into the arrangement of  $\text{dom } f^{\star 1}$ ;
6   end
7   foreach Dual Entity do
8     Compute its associated quadratic function;
9   end
10  Handle final special cases;
11  Return the PLQ2d function  $f^{\star 1}$ ;
12 end

```

can be done in log-linear time using a binary search tree or in linear time using some approximation and a hash table, see [19].

4.1 Dual entities associated with primal edges

The first step in the construction of the partial conjugate algorithm is to revise the decision tree for computing dual entities associated with primal edges. The full and partial conjugate trees have the same structure, though there are important rearrangements. See Figs. 2 and 3. The first decision we take is now whether or not an edge is horizontal. With the full conjugate, an edge along which the incident faces' gradients disagreed could expand along the x_2 direction to become a face, but this does not happen when building partial conjugate entities. If an edge is horizontal, then it is similar to the linear edge cases of the full conjugate, and even back to the 1D conjugate algorithm (a linear edge becomes a point; a quadratic edge becomes another quadratic edge). The dual entity associated with a primal horizontal edge will always be another horizontal edge (or, degenerately, a point).

If the edge is not horizontal, then we have parallels with the quadratic branch of the full conjugate tree. In this case, we must consider the points $\nabla_{10} f_i(\underline{x})$, $\nabla_{10} f_j(\underline{x})$, $\nabla_{10} f_i(\bar{x})$, and $\nabla_{10} f_j(\bar{x})$. The second, third, and optional fourth branch of the non-horizontal subtree all branch on the same conditions as the quadratic subtree. The resulting shapes differ, though: whereas a face associated with $+\infty$ in the quadratic subtree has for dual entity a ray pointing into the infinite face, orthogonal to the primal edge, such rays added to the domain of the partial conjugate point *horizontally* into the infinite face.

4.2 Partial conjugate function values

The full conjugate has every combination of primal vertices, edges, and faces associated with dual vertices, edges, and faces in the dual. There is much less variety in the

partial conjugate: a primal vertex cannot be associated with a dual face, for example. The only primal entities that are associated with a dual face are the non-horizontal edges (in certain cases), and the faces that are quadratic in x_1 . Thus, we need to compute conjugate function values for edges (horizontal and not), and for faces (that may be linear or quadratic w.r.t. x_1). We need not explicitly consider function values for vertices, as they will be covered by the edge cases (with the exception of special cases in Sect. 4.3). All the results in the following sections are extracted from the proofs in Sect. 3.

We will now write function values in the form

$$f(x_1, x_2) = a(x_1)^2 + bx_1x_2 + c(x_2)^2 + dx_1 + ex_2 + g.$$

4.2.1 Function values associated with edges

If an edge is horizontal, then x_2 is fixed, and we can eliminate it to get

$$f(x_1, x_2) = \tilde{a}x_1^2 + \tilde{b}x_1 + \tilde{c},$$

where $\tilde{a} = a$, $\tilde{b} = bx_2 + d$, $\tilde{c} = cx_2^2 + ex_2 + g$. Applying the explicit formula for the conjugate of a quadratic function gives

$$f^{*1}(s_1, x_2) = \begin{cases} \frac{1}{4\tilde{a}}(s_1 - \tilde{b})^2 - \tilde{c}, & \text{if } \tilde{a} > 0; \\ \iota_{\{\tilde{b}\}}(s_1) - \tilde{c}, & \text{if } \tilde{a} = 0. \end{cases}$$

For a non-horizontal edge, we first perform a change of variables to eliminate x_1 from $f(x_1, x_2)$ as much as possible, to simplify the computation. Suppose the edge lies on the line $x_1 = \alpha x_2 + \beta$, for some $\alpha, \beta \in \mathbb{R}$. The edge is associated with the function

$$ax_1^2 + bx_1x_2 + cx_2^2 + dx_1 + ex_2 + g + \iota_{\{\alpha x_2 + \beta\}}(x_1).$$

Substituting in $x_1 = \alpha x_2 + \beta$ gives

$$f(x_1, x_2) = \tilde{a}x_2^2 + \tilde{b}x_2 + \tilde{c} + \iota_{\{\alpha x_2 + \beta\}}(x_1),$$

where $\tilde{a} = \alpha^2 a + \alpha b + c$, $\tilde{b} = 2\alpha\beta a + \beta b + \alpha d + e$, $\tilde{c} = \beta^2 a + \beta d + g$. The quadratic function associated with the dual entity associated with this primal edge is now

$$\begin{aligned} f^{*1}(s_1, x_2) &= \sup_{x_1 \in \mathbb{R}} [s_1x_1 - \tilde{a}x_2^2 - \tilde{b}x_2 - \tilde{c} - \iota_{\{\alpha x_2 + \beta\}}(x_1)] \\ &= \sup_{x_1 \in \mathbb{R}} [s_1x_1 - \iota_{\{\alpha x_2 + \beta\}}(x_1)] - \tilde{a}x_2^2 - \tilde{b}x_2 - \tilde{c} \\ &= s_1(\alpha x_2 + \beta) - \tilde{a}x_2^2 - \tilde{b}x_2 - \tilde{c} \\ &= \begin{bmatrix} s_1 & x_2 \end{bmatrix} \begin{bmatrix} 0 & \alpha/2 \\ \alpha/2 & -\tilde{a} \end{bmatrix} \begin{bmatrix} s_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} \beta \\ -\tilde{b} \end{bmatrix}^T \begin{bmatrix} s_1 \\ x_2 \end{bmatrix} - \tilde{c}. \end{aligned}$$

4.2.2 Function values associated with faces

If a face is associated with a quadratic function that is linear in x_1 for fixed x_2 , i.e. $f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2 + dx_1 + ex_2 + g$ with $a = 0$, then this face conjugates to an edge on the line $s_1 = bx_2 + d$, with function value

$$\begin{aligned} f^{*1}(s_1, x_2) &= \sup_{x_1 \in \mathbb{R}} [s_1x_1 - bx_1x_2 - cx_2^2 - dx_1 - ex_2 - g] \\ &= \sup_{x_1 \in \mathbb{R}} [s_1x_1 - bx_1x_2 - dx_1] - cx_2^2 - ex_2 - g \\ &= \sup_{x_1 \in \mathbb{R}} [x_1(s_1 - bx_2 - d)] - cx_2^2 - ex_2 - g \\ &= -cx_2^2 - ex_2 - g + \iota_{\{bx_2+d\}}(s_1). \end{aligned}$$

If the function associated with the face is quadratic in x_1 ($a > 0$), then

$$\begin{aligned} f^{*1}(s_1, x_2) &= \sup_{x_1 \in \mathbb{R}} [s_1x_1 - ax_1^2 - bx_1x_2 - cx_2^2 - dx_1 - ex_2 - g] \\ &= \sup_{x_1 \in \mathbb{R}} [-ax_1^2 + (s_1 - bx_2 - d)x_1] - cx_2^2 - ex_2 - g. \end{aligned}$$

This supremum is maximized at $x_1 = \frac{1}{2a}(s_1 - bx_2 - d)$, and expanding gives

$$f^{*1}(s_1, x_2) = \frac{1}{4a} \begin{bmatrix} s_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & -b \\ -b & b^2 - 4ac \end{bmatrix} \begin{bmatrix} s_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} \frac{-d}{2a} \\ \frac{bd}{2a} - e \end{bmatrix}^T \begin{bmatrix} s_1 \\ x_2 \end{bmatrix} + \frac{d^2}{4a} - g.$$

4.3 Special cases

There are a few cases where it is not sufficient to consider only edges to construct the domain of a conjugate function. Line 2 of Algorithm 1 must still handle special cases (point-indicator and single-face functions), for which partial conjugates are easily computed.

There are three types of post-processing we must do in line 10. The first is to handle the case where edges are not sufficient to construct the conjugate domain. A primal face associated with a function that is linear in x_1 will be associated with a dual edge. The algorithm relies on some edge having a conjugate entity that includes the edges of any incident finite faces. Faces associated with a function that is linear in x_1 with no incident non-horizontal edges fail to meet this criteria (any incident horizontal edge will conjugate to a vertex). This case can be detected and handled by enumeration.

The second type of post-processing applies only to functions whose effective domains are a subset of a horizontal line. The conjugate of such a function is just the 1D conjugate of the function restricted to the line. If such a function's domain is bounded from the left, then the conjugate will have a linear edge extending to $x_1 = -\infty$, and likewise for boundedness from the right. Because there are no non-horizontal primal edges that would cover these conjugate edges, they must be inserted explicitly.

The third and final type of post-processing relates to the boundedness of the domain. For example, if there is a $z \in \mathbb{R}$ such that $f(x_1, x_2) = +\infty$ for all $x_2 > z, x_1 \in \mathbb{R}$, then $f^{*1}(s_1, x_2) = -\infty$ for all $x_2 > z, s_1 \in \mathbb{R}$. Partial conjugate functions may ultimately have both $+\infty$ and $-\infty$ -valued unbounded faces. The full conjugate algorithm sets any face outside of a function effective domain to $+\infty$, so this step must subdivide the $+\infty$ face(s), to create $-\infty$ faces for all x_2 where $f(x_1, x_2) = +\infty$, for all x_1 . If there is a z as above, then take z^* to be the minimum such z . If $f(x_1, z^*)$ is bounded either as $x_1 \rightarrow -\infty$ or $x_1 \rightarrow \infty$, then horizontal ray(s) should be inserted at $x_2 = z^*$ to split the infinite-valued face, and the $x_2 > z^*$ face should be assigned $-\infty$. Similar logic applies for a y such that $f(x_1, x_2) = \infty$ for all $x_2 < y, x_1 \in \mathbb{R}$.

The algorithm must also handle $-\infty$ while conjugating entities—if there is an $x_2 \in \mathbb{R}$ such that $f(x_1, x_2) = -\infty$ for some $x_1 \in \mathbb{R}$, then $f^{*1}(s_1, x_2) = \infty$ for all $s_1 \in \mathbb{R}$ with x_2 still fixed.

5 Conclusion

We proposed the first partial conjugate computation algorithm for bivariate convex PLQ functions. We compared it to the (full) conjugate computation algorithm implemented in [19], and provided several numerical examples. In addition, we recalled that the partial conjugate of a bivariate convex PLQ function is a convex-concave PLQ function, thereby extending the class of functions on which a PLQ calculus toolbox can be built.

The partial conjugate algorithm opens the door to computing partial versions of the Moreau envelope, proximal average, and other convex transforms that only operate on a single variable of a bivariate function.

An implementation issue that needs improvement is how equality between two PLQ2d functions is tested. Currently the test naïvely subtracts two functions and checks that the function values are equal on each entity in the resulting domain, not accounting for degenerate faces where the difference will not be zero (e.g. when two edges should be considered equal, but are not numerically so). One alternate approach would be, for each face in the overlaid domain, to select six points in general position that uniquely define a quadratic function, and compare the two functions at those six points only.

An interesting future work would be to consider extending the algorithm to functions of more than two variables using an approach similar to [35–37].

Acknowledgements Yves Lucet and Khan Jakee were partially supported by a Discovery grant from the Natural Sciences and Engineering Research Council of Canada (NSERC). Bryan Gardiner was partially supported by an NSERC Undergraduate Student Research Award. Part of the research was performed in the Computer-Aided Convex Analysis laboratory funded by the Canadian Foundation for Innovation.

Special thanks go to Rafal Goebel who pointed out the proof of Fact 3.3 in [20]. The authors are indebted to the referees for invaluable feedback that resulted in the inclusion of Fact 3.3, and greatly improved the presentation of the paper.

References

1. Alfeld, P., Schumaker, L.L.: Smooth macro-elements based on Powell-Sabin triangle splits. *Adv. Comput. Math.* **16**, 29–46 (2002)

2. Bauschke, H.H., Goebel, R., Lucet, Y., Wang, X.: The proximal average: basic theory. *SIAM J. Optim.* **19**, 768–785 (2008)
3. Bauschke, H.H., Lucet, Y., Trienien, M.: How to transform one convex function continuously into another. *SIAM Rev.* **50**, 115–132 (2008)
4. Bauschke, H.H., Lucet, Y., Wang, X.: Primal-dual symmetric intrinsic methods for finding antiderivatives of cyclically monotone operators. *SIAM J. Control Optim.* **46**, 2031–2051 (2007)
5. Bauschke, H.H., Moffat, S.M., Wang, X.: Self-dual smooth approximations of convex functions via the proximal average. In: Bauschke, H.H., Burachik, R.S., Combettes, P.L., Elser, V., Luke, D.R., Wolkowicz, H. (eds.) *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*. Springer Optimization and Its Applications, vol. 49, pp. 23–32. Springer, New York (2011)
6. Bauschke, H.H., Wang, X., Yao, L.: Autoconjugate representers for linear monotone operators. *Math. Program.* **123**, 5–24 (2010)
7. Brenier, Y.: Un algorithme rapide pour le calcul de transformées de Legendre–Fenchel discrètes. *C. R. Acad. Sci. Paris, Sér. I, Math.* **308**, 587–589 (1989)
8. Cao, J., Li, X., Wang, G., Qin, H.: Surface reconstruction using bivariate simplex splines on Delaunay configurations. In: *IEEE International Conference on Shape Modeling and Applications*. Computers & Graphics-UK, Tsinghua Univ, Beijing, P.R. China, Jun. 26–28, 2009, vol. 33, pp. 341–350 (2009)
9. CGAL: Computational geometry algorithms library. <http://www.cgal.org>
10. CGLAB: A Scilab toolbox for geometry based on CGAL. <http://cglab.gforge.inria.fr/>
11. Consortium, S.: Scilab (1994). <http://www.scilab.org>
12. Corrias, L.: Fast Legendre–Fenchel transform and applications to Hamilton–Jacobi equations and conservation laws. *SIAM J. Numer. Anal.* **33**, 1534–1558 (1996)
13. Dæhlen, M., Lyche, T.: Bivariate interpolation with quadratic box splines. *Math. Comput.* **51**, 219–230 (1988)
14. de Berg, M., van Kreveld, M., Overmars, M., Schwarzkopf, O.: *Computational Geometry*, 3rd edn. Algorithms and Applications. Springer, Berlin (2008)
15. Dierckx, P., Van Leemput, S., Vermeire, T.: Algorithms for surface fitting using Powell–Sabin splines. *IMA J. Numer. Anal.* **12**, 271–299 (1992)
16. Gardiner, B., Lucet, Y.: Numerical computation of Fitzpatrick functions. *J. Convex Anal.* **16**, 779–790 (2009)
17. Gardiner, B., Lucet, Y.: Convex hull algorithms for piecewise linear-quadratic functions in computational convex analysis. *Set-Valued Var. Anal.* **18**, 467–482 (2010)
18. Gardiner, B., Lucet, Y.: Graph-matrix calculus for computational convex analysis. In: Bauschke, H.H., Burachik, R.S., Combettes, P.L., Elser, V., Luke, D.R., Wolkowicz, H. (eds.) *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*. Springer Optimization and Its Applications, vol. 49, pp. 243–259. Springer, New York (2011)
19. Gardiner, B., Lucet, Y.: Computing the conjugate of convex piecewise linear-quadratic bivariate functions. *Math. Program.* 1–24 (2013)
20. Goebel, R.: Convexity, convergence and feedback in optimal control. PhD thesis, University of Washington, Seattle (2000)
21. Goebel, R.: Self-dual smoothing of convex and saddle functions. *J. Convex Anal.* **15**, 179–190 (2008)
22. Hare, W.: A proximal average for nonconvex functions: a proximal stability perspective. *SIAM J. Optim.* **20**, 650–666 (2009)
23. Hiriart-Urruty, J.-B., Lemaréchal, C.: *Convex Analysis and Minimization Algorithms*. Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences], vol. 305–306. Springer, Berlin (1993). Vol I: Fundamentals, Vol II: Advanced theory and bundle methods
24. Johnstone, J., Koch, V., Lucet, Y.: Convexity of the proximal average. *J. Optim. Theory Appl.* **148**, 107–124 (2011)
25. Lai, M.-J., Schumaker, L.L.: Spline Functions on Triangulations. *Encyclopedia of Mathematics and Its Applications*, vol. 110. Cambridge University Press, Cambridge (2007)
26. Lucet, Y.: A fast computational algorithm for the Legendre–Fenchel transform. *Comput. Optim. Appl.* **6**, 27–57 (1996)
27. Lucet, Y.: Computational convex analysis library, 1996–2011. <https://people.ok.ubc.ca/ylucet/ccca.html>
28. Lucet, Y.: Faster than the fast Legendre transform, the linear-time Legendre transform. *Numer. Algorithms* **16**, 171–185 (1997)
29. Lucet, Y.: Fast Moreau envelope computation I: numerical algorithms. *Numer. Algorithms* **43**, 235–249 (2006)

30. Lucet, Y.: What shape is your conjugate? A survey of computational convex analysis and its applications. *SIAM Rev.* **52**, 505–542 (2010)
31. Lucet, Y., Bauschke, H.H., Trienis, M.: The piecewise linear-quadratic model for computational convex analysis. *Comput. Optim. Appl.* **43**, 95–118 (2009)
32. Manni, C., Sablonnière, P.: Quadratic spline quasi-interpolants on Powell-Sabin partitions. *Adv. Comput. Math.* **26**, 283–304 (2007)
33. Moffat, S.M.: On the kernel average of n functions. Master's thesis, Department of Mathematics, University of British Columbia (Dec 2009)
34. Noullez, A., Vergassola, M.: A fast Legendre transform algorithm and applications to the adhesion model. *J. Sci. Comput.* **9**, 259–281 (1994)
35. Patrinos, P., Sarimveis, H.: Convex parametric piecewise quadratic optimization: theory, algorithms and control applications. Tech. rep., National Technical University of Athens, Greece (2010)
36. Patrinos, P., Sarimveis, H.: A new algorithm for solving convex parametric quadratic programs based on graphical derivatives of solution mappings. *Automatica* **46**, 1405–1418 (2010)
37. Patrinos, P., Sarimveis, H.: Convex parametric piecewise quadratic optimization: theory and algorithms. *Automatica* **47**, 1770–1777 (2011)
38. Penot, J.-P.: The relevance of convex analysis for the study of monotonicity. *Nonlinear Anal.* **58**, 855–871 (2004)
39. Powell, M.J.D., Sabin, M.A.: Piecewise quadratic approximations on triangles. *ACM Trans. Math. Softw.* **3**, 316–325 (1977)
40. Rockafellar, R.T.: *Convex Analysis*. Princeton University Press, Princeton (1970)
41. Rockafellar, R.T., Wets, R.J.-B.: *Variational Analysis*. Springer, Berlin (1998)
42. Sbibi, D., Serghini, A., Tijini, A.: Polar forms and quadratic spline quasi-interpolants on Powell-Sabin partitions. *Appl. Numer. Math.* **59**, 938–958 (2009)
43. Sbibi, D., Serghini, A., Tijini, A.: Bivariate simplex spline quasi-interpolants. *Numer. Math.* **3**, 97–118 (2010)
44. She, Z.-S., Aurell, E., Frisch, U.: The inviscid Burgers equation with initial data of Brownian type. *Commun. Math. Phys.* **148**, 623–641 (1992)
45. Sorokina, T., Zeilfelder, F.: Optimal quasi-interpolation by quadratic C^1 -splines on type-2 triangulations. In: *Approximation Theory XI: Gatlinburg 2004*, *Mod. Methods Math.*, pp. 423–438. Nashboro Press, Brentwood (2005)
46. Speleers, H., Dierckx, P., Vandewalle, S.: Quasi-hierarchical Powell-Sabin B-splines. *Comput. Aided Geom. Des.* **26**, 174–191 (2009)
47. Wiley, D.F., Childs, H.R., Hamann, B., Joy, K.I., Max, N.L.: Best quadratic spline approximation for hierarchical visualization. In: *Proceedings of the Symposium on Data Visualisation 2002, VISSYM '02*, Aire-la-Ville. Eurographics Association, Switzerland, pp. 133–140 (2002)
48. Willemans, K., Dierckx, P.: Surface fitting using convex Powell-Sabin splines. *J. Comput. Appl. Math.* **56**, 263–282 (1994)